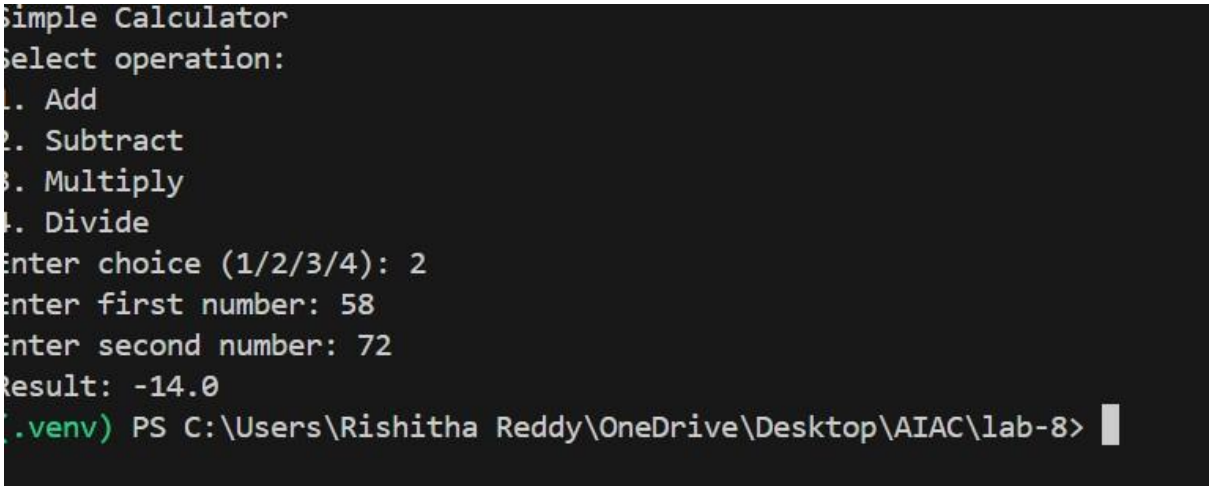
LAB-8

Task-0

```python
def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

def multiply(x, y):
    return x * y

def divide(x, y):
    if y == 0:
        return "Error: Division by zero"
    return x / y

def main():
    print("Simple Calculator")
    print("Select operation:")
    print("1. Add")
    print("2. Subtract")
    print("3. Multiply")
    print("4. Divide")

    choice = input("Enter choice (1/2/3/4): ")

    if choice in ('1', '2', '3', '4'):
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))

        if choice == '1':
```

```python
def main():
    print("Simple Calculator")
    print("Select operation:")
    print("1. Add")
    print("2. Subtract")
    print("3. Multiply")
    print("4. Divide")

    choice = input("Enter choice (1/2/3/4): ")

    if choice in ('1', '2', '3', '4'):
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))

        if choice == '1':
            print("Result:", add(num1, num2))
        elif choice == '2':
            print("Result:", subtract(num1, num2))
        elif choice == '3':
            print("Result:", multiply(num1, num2))
        elif choice == '4':
            print("Result:", divide(num1, num2))
    else:
        print("Invalid input")

if __name__ == "__main__":
    main()
```

OUTPUT:

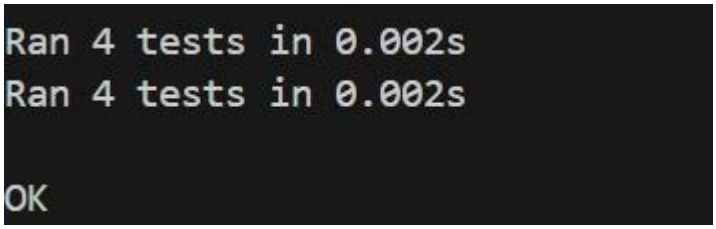

imple Calculator
elect operation:
. Add
. Subtract
. Multiply
. Divide
nter choice (1/2/3/4): 2
nter first number: 58
nter second number: 72
esult: -14.0
.venv) PS C:\Users\Rishitha Reddy\OneDrive\Desktop\AIAC\lab-8>

```python
import unittest
from Task0 import add, subtract, multiply, divide

class TestCalculator(unittest.TestCase):

    def test_add(self):
        self.assertEqual(add(2, 3), 5)
        self.assertEqual(add(-1, 1), 0)
        self.assertEqual(add(0, 0), 0)
        self.assertEqual(add(1.5, 2.5), 4.0)
        self.assertEqual(add(-3, -7), -10)
        self.assertEqual(add(1000000, 1), 1000001)
        self.assertAlmostEqual(add(-1.1, -2.2), -3.3, places=7)
        self.assertAlmostEqual(add(0.1, 0.2), 0.3, places=7)
        self.assertEqual(add(-100, 100), 0)

    def test_subtract(self):
        self.assertEqual(subtract(5, 3), 2)
        self.assertEqual(subtract(0, 5), -5)
        self.assertEqual(subtract(-2, -2), 0)
        self.assertEqual(subtract(10, 0), 10)
        self.assertEqual(subtract(-5, 5), -10)
        self.assertEqual(subtract(1.5, 0.5), 1.0)
        self.assertEqual(subtract(-10, -5), -5)
        self.assertAlmostEqual(subtract(0.3, 0.1), 0.2, places=7)
        self.assertEqual(subtract(1000000, 1), 999999)

    def test_multiply(self):
        self.assertEqual(multiply(4, 5), 20)
```

```python
def test_subtract(self):
    self.assertEqual(subtract(1000000, 1), 999999)

def test_multiply(self):
    self.assertEqual(multiply(4, 5), 20)
    self.assertEqual(multiply(-1, 3), -3)
    self.assertEqual(multiply(0, 100), 0)
    self.assertEqual(multiply(2.5, 4), 10.0)
    self.assertEqual(multiply(-2, -8), 16)
    self.assertEqual(multiply(1.5, -2), -3.0)
    self.assertAlmostEqual(multiply(1000, 0.001), 1.0, places=7)
    self.assertEqual(multiply(1, 0), 0)
    self.assertEqual(multiply(-1, 0), 0)

def test_divide(self):
    self.assertEqual(divide(10, 2), 5)
    self.assertEqual(divide(5, -1), -5)
    self.assertEqual(divide(0, 1), 0)
    self.assertEqual(divide(5, 0), "Error: Division by zero")
    self.assertEqual(divide(-8, 2), -4)
    self.assertEqual(divide(7, 2), 3.5)
    self.assertEqual(divide(-9, -3), 3)
    self.assertEqual(divide(0, -5), 0)
    self.assertEqual(divide(1.5, 0.5), 3.0)
    self.assertEqual(divide(0, 100), 0)
    self.assertEqual(divide(100, 0), "Error: Division by zero")
    self.assertAlmostEqual(divide(1, 3), 1/3, places=7)
```

```python
if __name__ == "__main__":
    unittest.main()
```

OUTPUT:



Task Description#1

Use AI to generate test cases for is_valid_email(email) and then implement the validator function.

Requirements:

- Must contain @ and . characters.
- Must not start or end with special characters.
- Should not allow multiple @.

```python
import unittest
import re

def is_valid_email(email):
    # Check for exactly one '@'
    if email.count('@') != 1:
        return False

    local, domain = email.split('@')

    # Must not be empty
    if not local or not domain:
        return False

    # Must not start or end with special characters
    if not local[0].isalnum() or not local[-1].isalnum():
        return False
    if not domain[0].isalnum() or not domain[-1].isalnum():
        return False

    # Domain must contain at least one dot, and TLD >= 2 chars
    if '.' not in domain:
        return False
    if len(domain.split('.')[-1]) < 2:
        return False

    # Allowed characters (alphanumeric, ., _, -)
    local_pattern = r'^[A-Za-z0-9][A-Za-z0-9._-]*[A-Za-z0-9]$'
    domain_pattern = r'^[A-Za-z0-9][A-Za-z0-9.-]*[A-Za-z0-9]$'

    if not re.match(local_pattern, local):
        return False
    if not re.match(domain_pattern, domain):
```

```
def is_valid_email(email):
    if not re.match(domain_pattern, domain):
        return False

    return True


class TestIsValidEmail(unittest.TestCase):
    def test_valid_emails(self):
        self.assertTrue(is_valid_email("user@example.com"))
        self.assertTrue(is_valid_email("john.doe@mail.co"))
        self.assertTrue(is_valid_email("a_b-c.d@domain.com"))
        self.assertTrue(is_valid_email("user123@sub.domain.org"

    def test_missing_at_or_dot(self):
        self.assertFalse(is_valid_email("userexample.com"))
        self.assertFalse(is_valid_email("user@examplecom"))
        self.assertFalse(is_valid_email("userexamplecom"))

    def test_multiple_at(self):
        self.assertFalse(is_valid_email("user@@example.com"))
        self.assertFalse(is_valid_email("user@ex@ample.com"))

    def test_starts_or_ends_with_special(self):
        self.assertFalse(is_valid_email(".user@example.com"))
        self.assertFalse(is_valid_email("user.@example.com"))
        self.assertFalse(is_valid_email("-user@example.com"))
        self.assertFalse(is_valid_email("user-@example.com"))
        self.assertFalse(is_valid_email("_user@example.com"))
        self.assertFalse(is_valid_email("user_@example.com"))
        self.assertFalse(is_valid_email("user@example.com."))
        self.assertFalse(is_valid_email("user@example.com-"))
        self.assertFalse(is_valid_email("user@example.com_"))
```

```
    def test_starts_or_ends_with_special(self):
        self.assertFalse(is_valid_email(".user@example.com"))
        self.assertFalse(is_valid_email("user.@example.com"))
        self.assertFalse(is_valid_email("-user@example.com"))
        self.assertFalse(is_valid_email("user-@example.com"))
        self.assertFalse(is_valid_email("_user@example.com"))
        self.assertFalse(is_valid_email("user_@example.com"))
        self.assertFalse(is_valid_email("user@example.com."))
        self.assertFalse(is_valid_email("user@example.com-"))
        self.assertFalse(is_valid_email("user@example.com_"))

    def test_empty_and_invalid(self):
        self.assertFalse(is_valid_email(""))
        self.assertFalse(is_valid_email("@."))
        self.assertFalse(is_valid_email("user@.com"))
        self.assertFalse(is_valid_email("user@com."))
        self.assertFalse(is_valid_email("user@domain"))     #
        self.assertFalse(is_valid_email("user@site.c"))     #


if __name__ == "__main__":
    unittest.main()
```

```python
import unittest
from Task1 import is_valid_email

class CustomTestIsValidEmail(unittest.TestCase):
    def test_valid_emails(self):
        self.assertTrue(is_valid_email("alice@example.com"))
        self.assertTrue(is_valid_email("bob.smith@domain.co"))
        self.assertTrue(is_valid_email("a_b-c.d@domain.com"))
        self.assertTrue(is_valid_email("user123@sub.domain.org"))

    def test_invalid_no_at(self):
        self.assertFalse(is_valid_email("alice.example.com"))
        self.assertFalse(is_valid_email("aliceexamplecom"))

    def test_invalid_no_dot(self):
        self.assertFalse(is_valid_email("alice@examplecom"))
        self.assertFalse(is_valid_email("alice@com"))

    def test_multiple_at(self):
        self.assertFalse(is_valid_email("alice@@example.com"))
        self.assertFalse(is_valid_email("alice@ex@ample.com"))

    def test_starts_or_ends_with_special(self):
        self.assertFalse(is_valid_email(".alice@example.com"))
        self.assertFalse(is_valid_email("alice.@example.com"))
        self.assertFalse(is_valid_email("-alice@example.com"))
        self.assertFalse(is_valid_email("alice-@example.com"))
        self.assertFalse(is_valid_email("_alice@example.com"))
        self.assertFalse(is_valid_email("alice_@example.com"))
        self.assertFalse(is_valid_email("alice@example.com."))
        self.assertFalse(is_valid_email("alice@example.com-"))
        self.assertFalse(is_valid_email("alice@example.com_"))
```

```python
def test_empty_and_invalid(self):
    self.assertFalse(is_valid_email(""))
    self.assertFalse(is_valid_email("@."))
    self.assertFalse(is_valid_email("alice@.com"))
    self.assertFalse(is_valid_email("alice@com."))

def test_invalid_characters(self):
    self.assertFalse(is_valid_email("alice!@example.com"))
    self.assertFalse(is_valid_email("alice$@example.com"))
    self.assertFalse(is_valid_email("alice@exa#mple.com"))
    self.assertFalse(is_valid_email("ali ce@example.com"))

    def test_tld_too_short(self):
        self.assertFalse(is_valid_email("alice@example.c"))
        self.assertFalse(is_valid_email("bob@domain.x"))

    def test_subdomains(self):
        self.assertTrue(is_valid_email("user@sub.domain.com")
        self.assertTrue(is_valid_email("user@a.b.c.domain.org

    def test_numeric_local_and_domain(self):
        self.assertTrue(is_valid_email("12345@67890.com"))
        self.assertTrue(is_valid_email("a1b2c3@1a2b3c.com"))

    def test_dot_and_dash_in_domain(self):
        self.assertTrue(is_valid_email("user@my-domain.com"))
        self.assertTrue(is_valid_email("user@my.domain.com"))
        self.assertFalse(is_valid_email("user@-domain.com"))
        self.assertFalse(is_valid_email("user@domain-.com"))
        self.assertFalse(is_valid_email("user@.domain.com"))
        self.assertFalse(is_valid_email("user@domain.com-"))
```

```
58          def test_dot_and_dash_in_domain(self):
59              self.assertTrue(is_valid_email("user@my-domain.com"))
60              self.assertTrue(is_valid_email("user@my.domain.com"))
61              self.assertFalse(is_valid_email("user@-domain.com"))
62              self.assertFalse(is_valid_email("user@domain-.com"))
63              self.assertFalse(is_valid_email("user@.domain.com"))
64              self.assertFalse(is_valid_email("user@domain.com-"))
65
66          def test_dot_and_dash_in_local(self):
67              self.assertTrue(is_valid_email("a.b-c_d@domain.com"))
68              self.assertFalse(is_valid_email(".abc@domain.com"))
69              self.assertFalse(is_valid_email("abc.@domain.com"))
70              self.assertFalse(is_valid_email("-abc@domain.com"))
71              self.assertFalse(is_valid_email("abc-@domain.com"))
72              self.assertFalse(is_valid_email("_abc@domain.com"))
73              self.assertFalse(is_valid_email("abc_@domain.com"))
74
75          def test_uppercase_emails(self):
76              self.assertTrue(is_valid_email("USER@EXAMPLE.COM"))
77              self.assertTrue(is_valid_email("User.Name@Domain.Co")
78
79  if __name__ == "__main__":
80      unittest.main()
```

OUTPUT:

```
.....
----------------------------------------------------------------------
Ran 5 tests in 0.001s

OK
(.venv) PS C:\Users\Rishitha Reddy\OneDrive\Desktop\AIAC\lab-8> & "C:/Users/Rishitha Reddy/OneDrive/Desktop/AIA
C/lab-8/.venv/Scripts/python.exe" "c:/Users/Rishitha Reddy/OneDrive/Desktop/AIAC/lab-8/test_Task1.py"
.......
----------------------------------------------------------------------
Ran 7 tests in 0.002s

OK
(.venv) PS C:\Users\Rishitha Reddy\OneDrive\Desktop\AIAC\lab-8>
```

Task Description#2 (Loops)

- Ask AI to generate test cases for assign_grade(score) function. Handle boundary and invalid inputs.

Requirements

- AI should generate test cases for assign_grade(score) where: 90-100: A, 80-89: B, 70-79: C, 60-69: D, <60: F
- Include boundary values and invalid inputs (e.g., -5, 105, "eighty").

```python
import unittest

def assign_grade(score):
    if not isinstance(score, (int, float)):
        return "Invalid input"
    if score < 0 or score > 100:
        return "Invalid input"
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"

class TestAssignGrade(unittest.TestCase):
    def test_grade_a(self):
        self.assertEqual(assign_grade(90), "A")
        self.assertEqual(assign_grade(95), "A")
        self.assertEqual(assign_grade(100), "A")

    def test_grade_b(self):
        self.assertEqual(assign_grade(80), "B")
        self.assertEqual(assign_grade(85), "B")
        self.assertEqual(assign_grade(89), "B")

    def test_grade_c(self):
        self.assertEqual(assign_grade(70), "C")
        self.assertEqual(assign_grade(75), "C")
        self.assertEqual(assign_grade(79), "C")
```

```
19  class TestAssignGrade(unittest.TestCase):
23          self.assertEqual(assign_grade(100), "A")
24
25      def test_grade_b(self):
26          self.assertEqual(assign_grade(80), "B")
27          self.assertEqual(assign_grade(85), "B")
28          self.assertEqual(assign_grade(89), "B")
29
30      def test_grade_c(self):
31          self.assertEqual(assign_grade(70), "C")
32          self.assertEqual(assign_grade(75), "C")
33          self.assertEqual(assign_grade(79), "C")
34
35      def test_grade_d(self):
36          self.assertEqual(assign_grade(60), "D")
37          self.assertEqual(assign_grade(65), "D")
38          self.assertEqual(assign_grade(69), "D")
39
40      def test_grade_f(self):
41          self.assertEqual(assign_grade(0), "F")
42          self.assertEqual(assign_grade(59), "F")
43          self.assertEqual(assign_grade(30), "F")
44
45      def test_invalid_inputs(self):
46          self.assertEqual(assign_grade(-5), "Invalid input")
47          self.assertEqual(assign_grade(105), "Invalid input")
48          self.assertEqual(assign_grade("eighty"), "Invalid input")
49          self.assertEqual(assign_grade(None), "Invalid input")
50          self.assertEqual(assign_grade([90]), "Invalid input")
51
52  if __name__ == "__main__":
53      unittest.main()
```

OUTPUT:

```
......
----------------------------------------------------------------------
Ran 6 tests in 0.001s

OK
```

```python
import unittest
from Task2 import assign_grade

class TestAssignGradeExternal(unittest.TestCase):
    def test_grade_a(self):
        self.assertEqual(assign_grade(90), "A")
        self.assertEqual(assign_grade(95), "A")
        self.assertEqual(assign_grade(100), "A")

    def test_grade_b(self):
        self.assertEqual(assign_grade(80), "B")
        self.assertEqual(assign_grade(85), "B")
        self.assertEqual(assign_grade(89), "B")

    def test_grade_c(self):
        self.assertEqual(assign_grade(70), "C")
        self.assertEqual(assign_grade(75), "C")
        self.assertEqual(assign_grade(79), "C")

    def test_grade_d(self):
        self.assertEqual(assign_grade(60), "D")
        self.assertEqual(assign_grade(65), "D")
        self.assertEqual(assign_grade(69), "D")

    def test_grade_f(self):
        self.assertEqual(assign_grade(0), "F")
        self.assertEqual(assign_grade(59), "F")
        self.assertEqual(assign_grade(30), "F")

    def test_invalid_inputs(self):
        self.assertEqual(assign_grade(-5), "Invalid input")
        self.assertEqual(assign_grade(105), "Invalid input")
        self.assertEqual(assign_grade("eighty"), "Invalid input")
```

```python
    def test_grade_c(self):
        self.assertEqual(assign_grade(70), "C")
        self.assertEqual(assign_grade(75), "C")
        self.assertEqual(assign_grade(79), "C")

    def test_grade_d(self):
        self.assertEqual(assign_grade(60), "D")
        self.assertEqual(assign_grade(65), "D")
        self.assertEqual(assign_grade(69), "D")

    def test_grade_f(self):
        self.assertEqual(assign_grade(0), "F")
        self.assertEqual(assign_grade(59), "F")
        self.assertEqual(assign_grade(30), "F")

    def test_invalid_inputs(self):
        self.assertEqual(assign_grade(-5), "Invalid input")
        self.assertEqual(assign_grade(105), "Invalid input")
        self.assertEqual(assign_grade("eighty"), "Invalid input")
        self.assertEqual(assign_grade(None), "Invalid input")
        self.assertEqual(assign_grade([90]), "Invalid input")

if __name__ == "__main__":
    unittest.main()
```

OUTPUT:

```
----------------------------------------------------------------------
Ran 6 tests in 0.001s

OK
(.venv) PS C:\Users\Rishitha Reddy\OneDrive\Desktop\AIAC\lab-8> & "C:/Users/Rishitha Reddy/OneDrive/Desktop/AIA
C/lab-8/.venv/Scripts/python.exe" "c:/Users/Rishitha Reddy/OneDrive/Desktop/AIAC/lab-8/test_Task2.py"
......
----------------------------------------------------------------------
```

Task Description#3

- Generate test cases using AI for is_sentence_palindrome(sentence). Ignore case, punctuation, and spaces

Requirement

- Ask AI to create test cases for is_sentence_palindrome(sentence)

(ignores case, spaces, and punctuation).

Task3.py > ...

```python
import unittest
import string

def is_sentence_palindrome(sentence):
    # Remove spaces and punctuation, convert to lowercase
    cleaned = ''.join(
        ch.lower() for ch in sentence if ch.isalnum()
    )
    return cleaned == cleaned[::-1]

class TestIsSentencePalindrome(unittest.TestCase):
    def test_true_palindromes(self):
        self.assertTrue(is_sentence_palindrome("A man a plan a ca
        self.assertTrue(is_sentence_palindrome("No lemon, no melo
        self.assertTrue(is_sentence_palindrome("Was it a car or a
        self.assertTrue(is_sentence_palindrome("Madam, in Eden, I
        self.assertTrue(is_sentence_palindrome("Able was I, I saw
        self.assertTrue(is_sentence_palindrome("Eva, can I see be
        self.assertTrue(is_sentence_palindrome("Red roses run no

    def test_false_palindromes(self):
        self.assertFalse(is_sentence_palindrome("This is not a pa
        self.assertFalse(is_sentence_palindrome("Hello, world!"))
        self.assertFalse(is_sentence_palindrome("Palindrome test

    def test_edge_cases(self):
        self.assertTrue(is_sentence_palindrome(""))
        self.assertTrue(is_sentence_palindrome("!@#$%^&*()"))  #
        self.assertTrue(is_sentence_palindrome("A"))           #
        self.assertTrue(is_sentence_palindrome("  "))          #

if __name__ == "__main__":
    unittest.main()
```

```
= cleaned[::-1]

Palindrome(unittest.TestCase):
lindromes(self):
rue(is_sentence_palindrome("A man a plan a canal Panama"))
rue(is_sentence_palindrome("No lemon, no melon"))
rue(is_sentence_palindrome("Was it a car or a cat I saw?"))
rue(is_sentence_palindrome("Madam, in Eden, I'm Adam"))
rue(is_sentence_palindrome("Able was I, I saw Elba"))
rue(is_sentence_palindrome("Eva, can I see bees in a cave?"))
rue(is_sentence_palindrome("Red roses run no risk, sir, on Nurse

alindromes(self):
alse(is_sentence_palindrome("This is not a palindrome"))
alse(is_sentence_palindrome("Hello, world!"))
alse(is_sentence_palindrome("Palindrome test sentence"))

ses(self):
rue(is_sentence_palindrome(""))
rue(is_sentence_palindrome("!@#$%^&*()"))  # Only punctuation
rue(is_sentence_palindrome("A"))           # Single character
rue(is_sentence_palindrome("  "))          # Only spaces
```

OUTPUT:

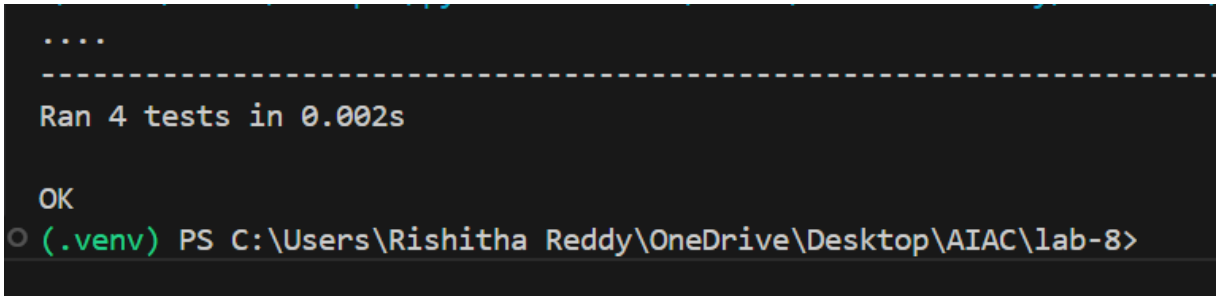
```
----------------------------------------------------------------------
Ran 3 tests in 0.000s

OK
(.venv) PS C:\Users\Rishitha Reddy\OneDrive\Desktop\AIAC\lab-8>
```

test_task3.py > ...

```python
import unittest
from Task3 import is_sentence_palindrome

class TestIsSentencePalindromeExternal(unittest.TestCase):
    def test_true_palindromes(self):
        self.assertTrue(is_sentence_palindrome("A man a plan a c
        self.assertTrue(is_sentence_palindrome("No lemon, no mel
        self.assertTrue(is_sentence_palindrome("Was it a car or
        self.assertTrue(is_sentence_palindrome("Madam, in Eden,
        self.assertTrue(is_sentence_palindrome("Able was I, I sa
        self.assertTrue(is_sentence_palindrome("Eva, can I see b
        self.assertTrue(is_sentence_palindrome("Red roses run no

    def test_false_palindromes(self):
        self.assertFalse(is_sentence_palindrome("This is not a p
        self.assertFalse(is_sentence_palindrome("Hello, world!")
        self.assertFalse(is_sentence_palindrome("Palindrome test

    def test_edge_cases(self):
        self.assertTrue(is_sentence_palindrome(""))
        self.assertTrue(is_sentence_palindrome("!@#$%^&*()"))  #
        self.assertTrue(is_sentence_palindrome("A"))           #
        self.assertTrue(is_sentence_palindrome("  "))          #

    def test_mixed_cases(self):
        self.assertTrue(is_sentence_palindrome("RaceCar"))
        self.assertTrue(is_sentence_palindrome("12321"))
        self.assertFalse(is_sentence_palindrome("12345"))
        self.assertTrue(is_sentence_palindrome("1a2b2a1"))
        self.assertFalse(is_sentence_palindrome("1a2b3a1"))

if __name__ == "__main__":
    unittest.main()
```

```
2   _palindrome
3
4   eExternal(unittest.TestCase):
5   (self):
6   ntence_palindrome("A man a plan a canal Panama"))
7   ntence_palindrome("No lemon, no melon"))
8   ntence_palindrome("Was it a car or a cat I saw?"))
9   ntence_palindrome("Madam, in Eden, I'm Adam"))
0   ntence_palindrome("Able was I, I saw Elba"))
1   ntence_palindrome("Eva, can I see bees in a cave?"))
2   ntence_palindrome("Red roses run no risk, sir, on Nurse's order."
3
4   s(self):
5   entence_palindrome("This is not a palindrome"))
6   entence_palindrome("Hello, world!"))
7   entence_palindrome("Palindrome test sentence"))
8
9   :
0   ntence_palindrome(""))
1   ntence_palindrome("!@#$%^&*()"))  # Only punctuation
2   ntence_palindrome("A"))           # Single character
3   ntence_palindrome("  "))          # Only spaces
4
5   ):
6   ntence_palindrome("RaceCar"))
7   ntence_palindrome("12321"))
8   entence_palindrome("12345"))
9   ntence_palindrome("1a2b2a1"))
0   entence_palindrome("1a2b3a1"))
1
```

OUTPUT:



Task Description#4

- Let AI fix it Prompt AI to generate test cases for a ShoppingCart class (add_item, remove_item, total_cost).

Methods:

Add_item(name,orice)

Remove_item(name)

Total_cost()

```python
import unittest

class ShoppingCart:
    def __init__(self):
        self.items = {}

    def add_item(self, name, price):
        if name in self.items:
            self.items[name].append(price)
        else:
            self.items[name] = [price]

    def remove_item(self, name):
        if name in self.items:
            self.items[name].pop()
            if not self.items[name]:
                del self.items[name]
            return True
        return False

    def total_cost(self):
        return sum(price for prices in self.items.values() for pr

class TestShoppingCart(unittest.TestCase):
    def setUp(self):
        self.cart = ShoppingCart()

    def test_add_single_item(self):
        self.cart.add_item("apple", 1.5)
        self.assertEqual(self.cart.total_cost(), 1.5)

    def test_add_multiple_items(self):
        self.cart.add_item("apple", 1.5)
```

```
class ShoppingCart:
    def total_cost(self):
        return sum(price for prices in self.items.values() for

class TestShoppingCart(unittest.TestCase):
    def setUp(self):
        self.cart = ShoppingCart()

    def test_add_single_item(self):
        self.cart.add_item("apple", 1.5)
        self.assertEqual(self.cart.total_cost(), 1.5)

    def test_add_multiple_items(self):
        self.cart.add_item("apple", 1.5)
        self.cart.add_item("banana", 2.0)
        self.assertEqual(self.cart.total_cost(), 3.5)

    def test_add_same_item_multiple_times(self):
        self.cart.add_item("apple", 1.5)
        self.cart.add_item("apple", 2.0)
        self.assertEqual(self.cart.total_cost(), 3.5)

    def test_remove_item(self):
        self.cart.add_item("apple", 1.5)
        self.cart.add_item("banana", 2.0)
        self.assertTrue(self.cart.remove_item("apple"))
        self.assertEqual(self.cart.total_cost(), 2.0)

    def test_remove_item_multiple_times(self):
        self.cart.add_item("apple", 1.5)
        self.cart.add_item("apple", 2.0)
        self.cart.remove_item("apple")
        self.assertEqual(self.cart.total_cost(), 1.5)
        self.cart.remove_item("apple")
```

```
class TestShoppingCart(unittest.TestCase):
    def test_remove_item(self):
        self.cart.add_item("apple", 1.5)
        self.cart.add_item("banana", 2.0)
        self.assertTrue(self.cart.remove_item("apple"))
        self.assertEqual(self.cart.total_cost(), 2.0)

    def test_remove_item_multiple_times(self):
        self.cart.add_item("apple", 1.5)
        self.cart.add_item("apple", 2.0)
        self.cart.remove_item("apple")
        self.assertEqual(self.cart.total_cost(), 1.5)
        self.cart.remove_item("apple")
        self.assertEqual(self.cart.total_cost(), 0.0)

    def test_remove_nonexistent_item(self):
        self.assertFalse(self.cart.remove_item("orange"))

    def test_total_cost_empty_cart(self):
        self.assertEqual(self.cart.total_cost(), 0.0)

if __name__ == "__main__":
    unittest.main()
```

OUTPUT:

```
Ran 7 tests in 0.001s

OK
(.venv) PS C:\Users\Rishitha Reddy\OneDrive\Desktop\AIAC\lab-8>
```

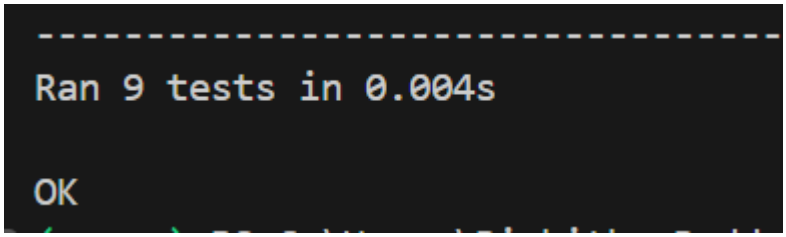
st_Task4.py > ShoppingCartUnitTests > setUp

```python
import unittest
from Task4 import ShoppingCart

class ShoppingCartUnitTests(unittest.TestCase):
    def setUp(self):
        self.cart = ShoppingCart()

    def test_add_single_item(self):
        self.cart.add_item("apple", 1.5)
        self.assertEqual(self.cart.items, {"apple": [1.5]})
        self.assertEqual(self.cart.total_cost(), 1.5)

    def test_add_multiple_items(self):
        self.cart.add_item("apple", 1.5)
        self.cart.add_item("banana", 2.0)
        self.assertEqual(self.cart.items, {"apple": [1.5], "banan
        self.assertEqual(self.cart.total_cost(), 3.5)

    def test_add_same_item_multiple_times(self):
        self.cart.add_item("apple", 1.5)
        self.cart.add_item("apple", 2.0)
        self.assertEqual(self.cart.items, {"apple": [1.5, 2.0]})
        self.assertEqual(self.cart.total_cost(), 3.5)

    def test_remove_item(self):
        self.cart.add_item("apple", 1.5)
        self.cart.add_item("banana", 2.0)
        result = self.cart.remove_item("apple")
        self.assertTrue(result)
        self.assertNotIn("apple", self.cart.items)
        self.assertEqual(self.cart.total_cost(), 2.0)

    def test_remove_item_multiple_times(self):
        self.cart.add_item("apple", 1.5)
```

```
        self.assertEqual(self.cart.total_cost(), 1.5)

    def test_add_multiple_items(self):
        self.cart.add_item("apple", 1.5)
        self.cart.add_item("banana", 2.0)
        self.assertEqual(self.cart.items, {"apple": [1.5], "banan
        self.assertEqual(self.cart.total_cost(), 3.5)

    def test_add_same_item_multiple_times(self):
        self.cart.add_item("apple", 1.5)
        self.cart.add_item("apple", 2.0)
        self.assertEqual(self.cart.items, {"apple": [1.5, 2.0]})
        self.assertEqual(self.cart.total_cost(), 3.5)

    def test_remove_item(self):
        self.cart.add_item("apple", 1.5)
        self.cart.add_item("banana", 2.0)
        result = self.cart.remove_item("apple")
        self.assertTrue(result)
        self.assertNotIn("apple", self.cart.items)
        self.assertEqual(self.cart.total_cost(), 2.0)

    def test_remove_item_multiple_times(self):
        self.cart.add_item("apple", 1.5)
        self.cart.add_item("apple", 2.0)
        self.assertTrue(self.cart.remove_item("apple"))
        self.assertEqual(self.cart.items, {"apple": [1.5]})
        self.assertEqual(self.cart.total_cost(), 1.5)
        self.assertTrue(self.cart.remove_item("apple"))
        self.assertEqual(self.cart.items, {})
        self.assertEqual(self.cart.total_cost(), 0.0)
```

```
    def test_remove_item_multiple_times(self):
        self.cart.add_item("apple", 1.5)
        self.cart.add_item("apple", 2.0)
        self.assertTrue(self.cart.remove_item("apple"))
        self.assertEqual(self.cart.items, {"apple": [1.5]})
        self.assertEqual(self.cart.total_cost(), 1.5)
        self.assertTrue(self.cart.remove_item("apple"))
        self.assertEqual(self.cart.items, {})
        self.assertEqual(self.cart.total_cost(), 0.0)

    def test_remove_nonexistent_item(self):
        self.assertFalse(self.cart.remove_item("orange"))

    def test_total_cost_empty_cart(self):
        self.assertEqual(self.cart.total_cost(), 0.0)

    def test_remove_item_from_empty_cart(self):
        self.assertFalse(self.cart.remove_item("apple"))

    def test_add_and_remove_multiple_items(self):
        self.cart.add_item("apple", 1.0)
        self.cart.add_item("banana", 2.0)
        self.cart.add_item("apple", 1.5)
        self.cart.remove_item("apple")
        self.assertEqual(self.cart.items, {"apple": [1.0], "banan
        self.cart.remove_item("banana")
        self.assertEqual(self.cart.items, {"apple": [1.0]})
        self.assertEqual(self.cart.total_cost(), 1.0)

f __name__ == "__main__":
    unittest.main()
```

OUTPUT:



Task Description#5

- Use AI to write test cases for convert_date_format(date_str) to switch from "YYYY-MM-DD" to "DD-MM-YYYY".

Example: "2023-10-15" → "15-10-2023"

Expected Output#5

- Function converts input format correctly for all test cases

- Task5.py > ...

```python
import unittest

def convert_date_format(date_str):
    try:
        parts = date_str.split('-')
        if len(parts) != 3:
            return "Invalid format"
        yyyy, mm, dd = parts
        if len(yyyy) != 4 or len(mm) != 2 or len(dd) != 2:
            return "Invalid format"
        return f"{dd}-{mm}-{yyyy}"
    except Exception:
        return "Invalid format"

class TestConvertDateFormat(unittest.TestCase):
    def test_valid_dates(self):
        self.assertEqual(convert_date_format("2023-10-15"), "15-1
        self.assertEqual(convert_date_format("2000-01-01"), "01-0
        self.assertEqual(convert_date_format("1999-12-31"), "31-1
        self.assertEqual(convert_date_format("2025-09-03"), "03-0

    def test_invalid_format(self):
        self.assertEqual(convert_date_format("2023/10/15"), "Inva
        self.assertEqual(convert_date_format("15-10-2023"), "Inva
        self.assertEqual(convert_date_format("2023-10"), "Invalid
        self.assertEqual(convert_date_format("20231015"), "Invali
        self.assertEqual(convert_date_format(""), "Invalid format
        self.assertEqual(convert_date_format("2023-1-5"), "Invali
        self.assertEqual(convert_date_format("abcd-ef-gh"), "Inva

    def test_edge_cases(self):
        self.assertEqual(convert_date_format("0000-00-00"), "00-0
        self.assertEqual(convert_date_format("9999-99-99"), "99-9
```

```
d_dates(self):
rtEqual(convert_date_format("2023-10-15"), "15-10-2023")
rtEqual(convert_date_format("2000-01-01"), "01-01-2000")
rtEqual(convert_date_format("1999-12-31"), "31-12-1999")
rtEqual(convert_date_format("2025-09-03"), "03-09-2025")

lid_format(self):
rtEqual(convert_date_format("2023/10/15"), "Invalid format")
rtEqual(convert_date_format("15-10-2023"), "Invalid format")
rtEqual(convert_date_format("2023-10"), "Invalid format")
rtEqual(convert_date_format("20231015"), "Invalid format")
rtEqual(convert_date_format(""), "Invalid format")
rtEqual(convert_date_format("2023-1-5"), "Invalid format")
rtEqual(convert_date_format("abcd-ef-gh"), "Invalid format")

_cases(self):
rtEqual(convert_date_format("0000-00-00"), "00-00-0000")
rtEqual(convert_date_format("9999-99-99"), "99-99-9999")
```

```
        self.assertEqual(convert_date_format("1999-12-31"), "31-
        self.assertEqual(convert_date_format("2025-09-03"), "03-

    def test_invalid_format(self):
        self.assertEqual(convert_date_format("2023/10/15"), "Inv
        self.assertEqual(convert_date_format("15-10-2023"), "Inv
        self.assertEqual(convert_date_format("2023-10"), "Invali
        self.assertEqual(convert_date_format("20231015"), "Inval
        self.assertEqual(convert_date_format(""), "Invalid forma
        self.assertEqual(convert_date_format("2023-1-5"), "Inval
        self.assertEqual(convert_date_format("abcd-ef-gh"), "Inv

    def test_edge_cases(self):
        self.assertEqual(convert_date_format("0000-00-00"), "00-
        self.assertEqual(convert_date_format("9999-99-99"), "99-

if __name__ == "__main__":
    unittest.main()
```

OUTPUT:

```
----------------------------------------------------------------
Ran 3 tests in 0.002s
```

```python
import unittest
from Task5 import convert_date_format

class TestConvertDateFormatExternal(unittest.TestCase):
    def test_valid_dates(self):
        self.assertEqual(convert_date_format("2023-10-15"), "15-1
        self.assertEqual(convert_date_format("2000-01-01"), "01-0
        self.assertEqual(convert_date_format("1999-12-31"), "31-1
        self.assertEqual(convert_date_format("2025-09-03"), "03-0

    def test_invalid_format(self):
        self.assertEqual(convert_date_format("2023/10/15"), "Inva
        self.assertEqual(convert_date_format("15-10-2023"), "Inva
        self.assertEqual(convert_date_format("2023-10"), "Invalid
        self.assertEqual(convert_date_format("20231015"), "Invali
        self.assertEqual(convert_date_format(""), "Invalid format
        self.assertEqual(convert_date_format("2023-1-5"), "Invali
        self.assertEqual(convert_date_format("abcd-ef-gh"), "Inva

    def test_edge_cases(self):
        self.assertEqual(convert_date_format("0000-00-00"), "00-0
        self.assertEqual(convert_date_format("9999-99-99"), "99-9

if __name__ == "__main__":
    unittest.main()
```

```
t convert_date_format

tDateFormatExternal(unittest.TestCase):
id_dates(self):
ertEqual(convert_date_format("2023-10-15"), "15-10-2023")
ertEqual(convert_date_format("2000-01-01"), "01-01-2000")
ertEqual(convert_date_format("1999-12-31"), "31-12-1999")
ertEqual(convert_date_format("2025-09-03"), "03-09-2025")

alid_format(self):
ertEqual(convert_date_format("2023/10/15"), "Invalid format")
ertEqual(convert_date_format("15-10-2023"), "Invalid format")
ertEqual(convert_date_format("2023-10"), "Invalid format")
ertEqual(convert_date_format("20231015"), "Invalid format")
ertEqual(convert_date_format(""), "Invalid format")
ertEqual(convert_date_format("2023-1-5"), "Invalid format")
ertEqual(convert_date_format("abcd-ef-gh"), "Invalid format")

e_cases(self):
ertEqual(convert_date_format("0000-00-00"), "00-00-0000")
ertEqual(convert_date_format("9999-99-99"), "99-99-9999")
```

OUTPUT:

```
----------------------------------------------------------------------
Ran 3 tests in 0.001s
```